Verifying Distributed Algorithms with Interactive Theorem Provers: A Survey of Approaches

MAXWELL MOIR, University of Canterbury, New Zealand

Formal verification is a topical field in modern Computer Science. Proving the correctness of programs is not only elegant in a theoretical sense, but is often necessary in mission-critical applications. Due to the difficulty in formalizing programs, reliability in industry is often left to automated testing. However, in specific circumstances where correctness must be ensured, formal proofs may be required.

CCS Concepts: • **Theory of computation → Logic and verification**.

Additional Key Words and Phrases: distributed systems, formal verification, proof assistants

ACM Reference Format:
Maxwell Moir. 2026. Verifying Distributed Algorithms with Interactive Theorem Provers: A Survey of Approaches. *J. ACM* 000, 0, Article 000 (2026), 5 pages. https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

In an increasingly interconnected world, distributed algorithms play an integral part in much of the software we use on a daily basis. Ensuring these algorithms function as intended protects against failures that may be technically, economically, or even medically, critical.

1.1 Motivation

Distributed systems are famously difficult to implement correctly. Their divided and concurrent nature can often lead to subtle bugs that are difficult to reproduce. Additionally, errors can propogate throughout a system, and partial failures can lead to unexpected negative outcomes. For this reason, it is often unrealistic to acheive total coverage with traditional testing. Formal verfication can provide a strong guarantee of correctness in these situations. Interactive theorem proving as a verification method is specificly strong for distributed systems, as it can verify correctness in all states without iteratively exploring them. Theorem proving also allows for flexibility in the paramaterization of systems, such as verifying an algorithm for any N nodes.

1.2 Contributions

For this reason, this survey will explore the verification of distributed algorithms with interactive theorem provers. It provides a taxonomy of different theorem-based verfication approaches and frameworks, reviews several common tools, and contrasts with other verification approaches. In addition, this survey will compare the theorem proving verification approach against other common methods, such as manual proofs and model checking.

Author's address: Maxwell Moir, University of Canterbury, Christchurch, New Zealand, mmo199@uclive.ac.nz.

ACM 0004-5411/2026/0-ART000
https://doi.org/XXXXXXX.XXXXXXX

2 METHODOLOGY

2.1 Literature Search Strategy

To identify relevant literature, this survey employed a systematic literature search strategy. Two major academic databases were selected, the ACM digital library and IEEE Xplore. These databases were chosen due to their broad coverage of research in fomral verification and distributed systems, and were accessed on April 17 2026.

To ensure the quality of the review, the PRISMA framework was used to inform the search strategy. The strategy consisted of constructing a query followed by database searching and screening. First, relevant keywords were curated based on the topic of this survey and a seach query was constructed. The query used is as follows.

> ("proof assistant" OR "theorem prover" OR "mechanized verification" OR "formal verification") AND ("distributed protocol" OR "distributed algorithm")

Selected papers were limited to peer-reviewed English language publications. There was no strict lower bound imposed on the publication date, although results were naturally concentrated in the last 20 years. This query yeilded 213 results from the ACM Digital Library and 49 results from IEEE Xplore, which were exported for further screening.

2.2 Inclusion and Exclusion Criteria

These results were then screened by title and abstract with standard inclusion and exclusion requirements. Included papers focused on verfication of distributed algorithms using theorem proving. Duplicate papers within and between databases were removed. Additionally, papers were excluded if model checking or manual proof were the primary verification technique. This reduced the number of results to 37 for ACM and 15 for IEEE Xplore.

2.3 Citation Chaining

Finally, the reference lists of the selected papers were scanned and fundamental or commonly referenced papers were added. Google scholar was used as a supplementary tool for citation chaining. Any additional papers identified this way were included only if they met the above criteria, or primarily concerned a related topic to be explained.

3 PRELIMINARY

3.1 Distributed Algorithms

Distributed computing is a fundamental field in Computer Science. It concerns the inter-communication between distinct autonomous components in a system. This patterns shows up often, usually when there is a network of multiple devices.

There are several common challenges in distributed computing that are often the targets of formal verification. For example, consensus protocols such as Paxos Another example is Byzantine Fault Tolerance.

3.2 Interactive Theorem Provers

Interactive Theorem Provers, or proof assistants, have seen a recent surge in popularity among computer scientists and mathematicians alike. Proof assistants are tools used to assist with the human development of formal proofs. These formal proofs can then be automatically checked by the machine, ensuring correctness at each stage of the process. Theorem provers are extremely useful in the formal verification of protocols, where they can be used to ensure specific properties of a sytem hold. For this reason proof assistants such as Coq, Lean, Isabelle, and Dafny have become popular for verifying distributed algorithms.

3.3 Verification

Verifying the correctness of a program requires that specific conditions about a system can be shown to hold in certain states. These conditions can be arbitrairily chosen, but usually represent ... The two most notable property categories of a system are "Safety" and "Liveness". A safety property ensures protection against unfavourable events, whereas liveness properties make sure that the program functions as intended.

Fault tolerance is another notable aspect of ensuring a program works as intended. Byzantine fault tolerance, specifically, is a common problem in

4 APPROACHES TO VERIFYING DISTRIBUTED ALGORITHMS

4.1 Proof techniques

Inductive invariants Temporal logic based Refinement based Compositional [15] Automation

4.2 Tools and Common frameworks

There are many languages and frameworks used for the verification of distributed systems. The most commonly used tool in the literature is Rocq, formerly Coq. There are multiple frameworks implemented for Rocq that specifically focus on distributed systems, such as Verdi [14], and Diesel [11]. These ...

Another theorem prover TLAPS [2]

Iron Fleet [5]

A more contemporary tool is Lean4. There is less research in verification with lean4, as it is a younger tool. In 2025, the Veil [8] framework was introduced as a basis to verify distributed protocols specifically.

5 CASE STUDIES

5.1 Consensus Algorithms

Consensus algorithms are used to agree on one result in a group of multiple, possibly unreliable, participants. Two well known solutions to the consensus problem are the Paxos and Raft protocols. The Paxos protocol, originally specified by Lamport [4], is one method to facillitate consensus.

Consensus is an important problem in application too. For example, in blockchain systems consensus is required between distributed, and potentially malicious, machines to ensure users can agree on the state of the system. In their 2018 paper [9], Pirlea and Sergey present blockchain-based distributed consensus protocol, verified in the Rocq proof assistant. This is an example of a modern application of a formally verified distributed protocol, in which correctness is absolutely critical for user trust in the system.

5.2 Byzantine Fault Tolerance

Another fundamental problem in distributed computing is Byzantine Fault Tolerance. This problem was also introduced by Lamport et. al [6], and addresses partial malfunction in a distributed system. The problem

BFT [15]

Once again, this problem arises in blockchain systems. Blockchain Byzantine [12]

5.3 Mutual Exclusion

Mutual exclusion is another classic distributed computing problem. It was
[14]

5.4 Verified software

AWS [1] Mongo [10]

6 COMPARISON OF VERIFICATION STRATEGIES

There are three major verification paradigms that are often used for distributed algorithm verification. The other two predominant methods of formally verifying distributed systems are using model checking and manual proofs. Each of these other verification approaches has its own advantages and disadvantages in relation to theorem proving.

Many solutions to the previously mentioned problems are based on manual proofs. For example, De Prisco et al. [3] developed a I/O automaton model and verify the PAXOS algorithm. The paper contains a manual correctness proof, with the correctness of each component proved with invariants. Manual proofs are more expressive than mechanized proofs, but have a weaker correctness guarantee because depend on mathematical arguments formulated by humans. This means they are more prone to errors in their development or implementation, and are more difficult maintain when scaling.

Model checking is another approach for verifying the correctness of a distributed system. Model checking expands the state space, exhaustively exploring every possible state and ensuring that the correctness properties hold in each one. This was exemplified by Tsuchiya and Springer [13], who used bounded model checking to verify two consensus algorithms, including Hybrid-1, an improved version of the Fast Paxos algorithm. They were able to mechanically verify agreement up to 8 proccesses and termination up to 11 processes. A major limitation of model checking is the state space explosion problem, which restricts it to bounded or specifically abstracted models. This often reduces the maximum number of processes in a verified protocol.

A theorem proving approach does not have this issue, as it can ensure that correctness conditions hold for any finite number of processes.

7 CHALLENGES AND FUTURE DIRECTIONS

One of the common criticisims of theorem based formal verification is that it is too time consuming to apply practically.

[7] Proof effort Scalability There are differing perspectives on this .

Increased automation Machine learning

8 CONCLUSION

This paper aims to provide an overview of different techniques and problems in the formal verfication of distributed systems using theorem proving.

Interactive theorem provers provide the strongest guarantee of correctness, but are still expensive when compared to other methods.

REFERENCES

[1] Marc Brooker and Ankush Desai. 2025. Systems Correctness Practices at Amazon Web Services. *Commun. ACM* 68, 6 (June 2025), 38–42. https://doi.org/10.1145/3729175

[2] Denis Cousineau, Damien Doligez, Leslie Lamport, Stephan Merz, Daniel Ricketts, and Hernan Vanzetto. 2012. TLA+ Proofs. *Proceedings of the 18th International Symposium on Formal Methods (FM 2012), Dimitra Giannakopoulou and Dominique Mery, editors. Springer-Verlag Lecture Notes in Computer Science* 7436 (Jan. 2012), 147–154.

[3] Roberto De Prisco, Butler Lampson, and Nancy Lynch. 1997. Revisiting the Paxos Algorithm. In *Distributed Algorithms*, Gerhard Goos, Juris Hartmanis, Jan Van Leeuwen, Marios Mavronicolas, and Philippas Tsigas (Eds.). Vol. 1320. Springer Berlin Heidelberg, Berlin, Heidelberg, 111–125. https://doi.org/10.1007/BFb0030679

[4] Digital Equipment Corporation and Leslie Lamport. 2019. *The Part-Time Parliament.* Association for Computing Machinery. https://doi.org/10.1145/3335772.3335939

[5] Chris Hawblitzel, Jon Howell, Manos Kapritsos, Jacob R. Lorch, Bryan Parno, Michael L. Roberts, Srinath Setty, and Brian Zill. 2015. IronFleet: Proving Practical Distributed Systems Correct. In *Proceedings of the 25th Symposium on Operating Systems Principles (SOSP '15)*. Association for Computing Machinery, New York, NY, USA, 1–17. https://doi.org/10.1145/2815400.2815428

[6] Leslie Lamport, Robert Shostak, and Marshall Pease. 2019. The Byzantine Generals Problem. In *Concurrency: The Works of Leslie Lamport*. Association for Computing Machinery, New York, NY, USA, 203–226.

[7] Junrui Liu, Yanju Chen, Bryan Tan, Isil Dillig, and Yu Feng. 2023. Learning Contract Invariants Using Reinforcement Learning. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE '22)*. Association for Computing Machinery, New York, NY, USA, 1–11. https://doi.org/10.1145/3551349.3556962

[8] George Pîrlea, Vladimir Gladshtein, Elad Kinsbruner, Qiyuan Zhao, and Ilya Sergey. 2025. Veil: A Framework for Automated and Interactive Verification of Transition Systems. In *Computer Aided Verification: 37th International Conference, CAV 2025, Zagreb, Croatia, July 23-25, 2025, Proceedings, Part III*. Springer-Verlag, Berlin, Heidelberg, 26–41. https://doi.org/10.1007/978-3-031-98682-6_2

[9] George Pîrlea and Ilya Sergey. 2018. Mechanising Blockchain Consensus. In *Proceedings of the 7th ACM SIGPLAN International Conference on Certified Programs and Proofs*. ACM, Los Angeles CA USA, 78–90. https://doi.org/10.1145/3167086

[10] William Schultz, Ian Dardik, and Stavros Tripakis. 2022. Formal Verification of a Distributed Dynamic Reconfiguration Protocol. In *Proceedings of the 11th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2022)*. Association for Computing Machinery, New York, NY, USA, 143–152. https://doi.org/10.1145/3497775.3503688

[11] Ilya Sergey, James R. Wilcox, and Zachary Tatlock. 2017. Programming and Proving with Distributed Protocols. *Proc. ACM Program. Lang.* 2, POPL (Dec. 2017), 28:1–28:30. https://doi.org/10.1145/3158116

[12] Pierre Tholoniat and Vincent Gramoli. 2022. Formal Verification of Blockchain Byzantine Fault Tolerance. In *Handbook on Blockchain*, Duc A. Tran, My T. Thai, and Bhaskar Krishnamachari (Eds.). Vol. 194. Springer International Publishing, Cham, 389–412. https://doi.org/10.1007/978-3-031-07535-3_12

[13] Tatsuhiro Tsuchiya and André Schiper. 2008. Using Bounded Model Checking to Verify Consensus Algorithms. In *Proceedings of the 22nd International Symposium on Distributed Computing (DISC '08)*. Springer-Verlag, Berlin, Heidelberg, 466–480. https://doi.org/10.1007/978-3-540-87779-0_32

[14] James R. Wilcox, Doug Woos, Pavel Panchekha, Zachary Tatlock, Xi Wang, Michael D. Ernst, and Thomas Anderson. 2015. Verdi: A Framework for Implementing and Formally Verifying Distributed Systems. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '15)*. Association for Computing Machinery, New York, NY, USA, 357–368. https://doi.org/10.1145/2737924.2737958

[15] Qiyuan Zhao, George Pîrlea, Karolina Grzeszkiewicz, Seth Gilbert, and Ilya Sergey. 2024. Compositional Verification of Composite Byzantine Protocols. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security (CCS '24)*. Association for Computing Machinery, New York, NY, USA, 34–48. https://doi.org/10.1145/3658644.3690355